

Dynamic Programming Without Guessing

Dynamic programming reuses solutions to repeated subproblems whose relevant history can be represented by a state. Start with the meaning of a state and the final answer, then derive transitions and boundaries.

Memorizing a table shape without this meaning makes variants difficult to solve.

For: Candidates who can follow a DP solution after reading it but struggle to define the state, recurrence, base cases, and iteration order independently.

Suggested pace: 3 weeks of state-first practice | Recursion and basic complexity understood

What you will be able to do

- Define a state that contains exactly the information future decisions need.
- Derive transitions, base cases, and valid evaluation order from that state.
- Distinguish counting, feasibility, minimum-cost, and maximum-value objectives.
- Optimize memory only after explaining dependencies and preserving correctness.

Build the right mental model

State is a sufficient summary

If two histories produce the same state, their future options and objective contribution must be equivalent under the recurrence. Missing a remaining budget, last choice, or position can merge subproblems that actually differ. Too much history creates an unnecessarily large state space.

Base cases define semantics

An empty prefix, zero remaining amount, or completed target often has a special value. For counting, one empty construction can be correct; for minimization, unreachable states need a sentinel. A wrong base case can make a plausible recurrence consistently wrong.

Order enforces reuse rules

In-place updates can accidentally reuse an item more than once or count permutations instead of combinations. Determine whether a transition reads the previous layer or current layer before choosing loop order or compressing a table.

Study with evidence.

For every problem, write four sentences before code: state meaning, transition, base cases, and answer location. Begin with memoized recursion when it clarifies the model, then derive bottom-up order and account for recursion depth.

Week 1, first half: One-dimensional decisions

Practice stairs, non-adjacent selection, and minimum coin count. Draw the recursion tree and identify repeated states. Distinguish the value returned by a state from the decision path used to reconstruct a solution.

Deliverable and check: A recurrence with explicit unreachable and empty-input behavior, checked against brute force on small cases.

Week 1, second half: Counting and reuse

Compare subset sum, unbounded coin combinations, and ordered compositions. Trace updates for a tiny target and explain how loop direction controls reuse. Do not transfer a loop nest from one contract without proof.

Deliverable and check: A small example where two superficially similar loop orders intentionally produce different counts.

Week 2: Two-dimensional states

Study sequence alignment, edit distance, and grid paths. Define what each prefix index represents and which earlier states a cell depends on. Initialize empty rows and columns according to the actual operations.

Deliverable and check: A filled small table whose every transition can be explained in words.

Week 3: Optimize and generalize

Practice interval or constrained states relevant to your level. Estimate state count times transition cost. Compress memory only after proving overwritten values are no longer needed, then run unseen mixed variants.

Deliverable and check: A complete solution with a correctness argument, complexity, and tests that detect update-order mistakes.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Count combinations, not orderings

With unlimited positive coin values 1 and 2, count ways to make amount 3 when order does not matter. The valid combinations are 1+1+1 and 1+2, so the answer is two rather than three.

A worked approach

1. Define `ways[amount]` as the number of combinations using coin types processed so far. Initialize `ways[0]` to one for the empty construction, and all positive amounts to zero.
2. Process coin 1 across amounts in increasing order, giving one way for every amount through 3. Then process coin 2: `ways[2]` gains `ways[0]`, and `ways[3]` gains `ways[1]`. The final `ways[3]` is two.
3. Explain the alternative loop order: iterating target amounts outside and choosing the last coin inside counts ordered sequences, so 1+2 and 2+1 become different. Neither order is universally correct; the requested meaning decides.

Expected result

The recurrence and loop order now follow a precise counting contract. For c coin types and target A , this version takes $O(c \cdot A)$ time and $O(A)$ auxiliary space, with positive coin values assumed.

Verify the result

- Does amount zero produce one empty combination?
- Are repeated coin denominations normalized or otherwise handled according to the input contract?
- Does the tiny amount-three example distinguish combinations from sequences?

Extend the exercise

Change the problem so each coin instance can be used only once. Explain why a backward amount traversal prevents reading a value updated by the same item during the current iteration.

Make the learning stick.

How do I recognize a DP problem?

Look for choices leading to repeated subproblems whose future depends on a compact state. Recursion alone is not enough: backtracking with mostly unique states may not benefit from memoization, and a greedy proof may eliminate the need for DP.

What is the complexity of memoization?

Count reachable states and the work per state, including loops, copying, and lookup costs. Also account for stored results and recursion depth. Saying $O(n)$ because each function is cached can ignore many transitions.

Should I always compress to one row?

No. First obtain a correct state model and dependency order. A clear full table can be preferable during reasoning, especially if reconstruction is required. Compression is useful only when it preserves the needed previous values.

Avoid these traps

- Writing a recurrence without defining what a cell means.
- Using zero for both unreachable and valid zero-cost states.
- Changing loop order without checking item reuse or counting semantics.

Ready for the next step?

- State, transition, boundaries, and answer location are explicit.
- Small cases agree with an independent brute-force method.
- Complexity counts both states and transitions.
- Memory optimization has a dependency argument.

Reliable next reads

- [MIT algorithms course](#)
- [CP-Algorithms: dynamic programming](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.